InsPIRe: Scheme Specification and Error Analysis

Implementation Spec for `inspire-gpu` (C++/CUDA)

Keewoo Lee

May 2026

1 Overview

This document is the implementation specification for the InsPIRe PIR scheme [1], targeting a C++/CUDA library that runs on the GPU. All algorithms are given as pseudocode at implementation-ready detail.

InsPIRe is a PIR protocol with three computational steps:

1. **Matrix–vector product:** selects the target row, producing scalar LWE intermediates.
2. **InspiRING ring packing:** converts n scalar intermediates into one RLWE ciphertext.
3. **RGSW polynomial evaluation:** Horner's method with an RGSW query selects the target column group, reducing D packed ciphertexts to one.

The response consists of $c = \texttt{db\_cols}/(n \cdot D)$ compressed RLWE ciphertexts, each encrypting n plaintext values.

`inspire-gpu` implements this scheme in C++/CUDA on the GPU, using custom kernels for the matrix–vector product, NTT, gadget decomposition, ring packing collapse, and external products. Both RNS primes fit in 28 bits, so all device arithmetic uses 32-bit modular operations with Barrett reduction; 64-bit accumulators hold pre-reduction products.

Section 11 tracks the optimizations introduced in this implementation relative to the reference Rust implementation [1].

Scope: univariate evaluation. This document and the current `inspire-gpu` implementation describe a *univariate* Horner evaluation in step 3: the column-group index is encoded along a single axis as a polynomial of degree $D-1$ evaluated at one point ω^{j^*}, requiring one RGSW ciphertext in the query. A *multivariate* extension—factoring the column-group index into r axes of degree $D^{1/r}$ and evaluating an r-variate polynomial via nested Horner with r RGSWs—is a planned future direction. Multivariate evaluation reduces noise growth (multiplicative depth $r \cdot \log D^{1/r}$ vs. $\log D$ becomes irrelevant; what matters is the lower per-axis degree) and would raise $D_{\max}$, lowering precomputation memory at fixed database size. Extending the scheme would touch the index decomposition (Algorithm 19), HornerEval (Algorithm 21), the database encoding (Algorithm 18 becomes a separable r-dimensional IDFT), and the noise bound (Section 8.3).

2 Notation

3 Parameters

The packing uses two families of automorphisms: forward (τ_{5^k}) and conjugate (τ_{2n-5^k}), where 5 generates $(\mathbb{Z}/2n\mathbb{Z})^*/\{\pm 1\}$ of order $n/2$, and $\tau_{2n-1} : X \mapsto X^{-1}$ is the conjugation automorphism. Concrete parameter values are given in Section 9.3.

Table 1: Notation conventions.

Symbol	Meaning
n	Lattice dimension / polynomial degree (power of two)
N	Number of entries in the database
w	Size of each database entry in bytes
$q = q_0 \cdot q_1$	Ciphertext modulus (product of two NTT-friendly primes)
p	Plaintext modulus
$\Delta = \lfloor q/p \rfloor$	Scale factor
σ	Discrete Gaussian standard deviation
χ_{err}	Error distribution: each coefficient i.i.d. from $D_{\mathbb{Z},\sigma}$
χ_{sk}	Secret key distribution: uniform ternary $\{-1, 0, +1\}$ per coefficient
d	Number of gadget digits; $d-1$ effective (approximate)
B	Gadget base
D	Interpolation degree
$\mathcal{R}_q$	Ring $\mathbb{Z}_q[X]/(X^n + 1)$
$\mathcal{R}_p$	Ring $\mathbb{Z}_p[X]/(X^n + 1)$
τ_k	Automorphism: $\tau_k(f(X)) = f(X^k) \bmod (X^n + 1)$
$[\cdot]_q$	Reduction modulo q to $(-q/2, q/2]$

Discrete Gaussian sampling. We sample from the discrete Gaussian distribution χ_{err}: for integer x, $\Pr[X = x] \propto \exp(-x^2/2\sigma^2)$.

Implementation. We use the *cumulative distribution table* (CDT) method following OpenFHE: precompute the CDF of the discrete Gaussian for $|x| = 0, 1, \ldots, \lfloor 6\sigma \rceil$. To sample, draw a uniform random value and binary-search the table; the sign is determined by a random bit.

4 Cryptographic Primitives

4.1 Ring Arithmetic and NTT

All polynomial arithmetic is in $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n + 1)$. Polynomials are arrays of n integers in $[0, q)$. In RNS form, $q = q_0 \cdot q_1$; each polynomial is stored as two limbs of n values (one per prime). All element-wise and NTT operations are performed independently per limb.

Number Theoretic Transform. Ring multiplication uses the NTT: $a(X) \cdot b(X) = \text{INTT}(\text{NTT}(a) \circ \text{NTT}(b))$, where $\circ$ is coefficient-wise multiplication in the NTT domain. The NTT is a length-n radix-2 Cooley–Tukey transform over $\mathbb{Z}_{q_c}$ using a primitive $2n$-th root of unity $\psi_c \in \mathbb{Z}_{q_c}$ (exists because $q_c \equiv 1 \pmod{2n}$). Specifically, the "negacyclic NTT" premultiplies by powers of ψ_c to absorb the $X^n + 1$ reduction:

$$\hat{f}[i] = \sum_{j=0}^{n-1} f[j] \cdot \psi_c^j \cdot \omega_c^{ij} \pmod{q_c}, \qquad \omega_c = \psi_c^2$$

where ω_c is a primitive n-th root of unity. The INTT inverts this with $n^{-1} \bmod q_c$ scaling.

Representation convention. Ciphertexts and keys are stored in **NTT form** by default (both limbs). Conversion to coefficient form (INTT) is performed only when needed: LWE extraction, gadget decomposition, automorphism application, and serialization.

Table 2: Scheme parameters.

Parameter	Symbol	Description
Lattice dimension	n	Polynomial degree (power of two)
Ciphertext modulus	$q = q_0 \cdot q_1$	Product of two NTT-friendly primes ($q_i \equiv 1 \pmod{2n}$)
Plaintext modulus	p	Odd; each coefficient encodes $\lfloor \log_2 p \rfloor$ bits
Scale factor	$\Delta = \lfloor q/p \rfloor$	
Noise std. dev.	σ	Discrete Gaussian
Secret key dist.	χ_{sk}	Uniform ternary $\{-1, 0, +1\}$ per coefficient
Gadget digits	d	Number of digits; $d-1$ effective (approximate)
Gadget base	$B = 2^{\lceil \log_2 q/d \rceil}$	
Interpolation degree	D	Power of two; $D \leq 2n$, bounded by noise budget (Section 8.3)
Response compression (modulus switching)		
b modulus	q_0'	Bits per b coefficient on wire
a modulus	q_1'	Bits per a coefficient on wire

4.2 RLWE Encryption

Algorithm 1 SKGen()

1: Sample $s \in \mathcal{R}_q$: each coefficient from χ_{sk}
2: **return** s

Algorithm 2 RLWE.Dec(s, ct): Decrypt.

1: $(a, b) \leftarrow \mathsf{ct}$
2: $v(X) \leftarrow [b(X) - a(X) \cdot s(X)]_q$ ▷ centered reduction
3: **return** $\lfloor v(X) \cdot p/q \rceil \bmod p$ ▷ coefficient-wise

4.3 LWE Structure

An RLWE ciphertext $(a(X), b(X))$ with $b(X) = a(X) \cdot s(X) + e(X) + \Delta \cdot m(X)$ implicitly contains n scalar LWE ciphertexts, one per coefficient. The b-component of the j-th LWE ciphertext is simply $b[j]$ (the j-th coefficient of $b(X)$). The a-component is the j-th row of the *negacyclic matrix* of $a(X)$ — reconstructible from the seed by the server.

Key observation: LWE.Enc$_b$ (Algorithm 3) only needs to output the b-coefficients. No explicit LWE extraction is required on the client side.

Remark 1 (Correctness)**.** *Write $v(X) = b(X) - a(X) \cdot s(X)$ in $\mathcal{R}_q$. The j-th coefficient of the product $a \cdot s$ (whose j-th coefficient is subtracted from $b[j]$) modulo $X^n + 1$ is:*

$$(a \cdot s)_j = \sum_{k=0}^{j} a[j-k]\, s[k] \; - \sum_{k=j+1}^{n-1} a[n+j-k]\, s[k] \tag{1}$$

where the sign flip comes from $X^n \equiv -1$. This equals $\langle \mathbf{a}_j, \mathbf{s} \rangle$ with the $\mathbf{a}_j$ above. Hence $v_j = \mathsf{ct}_j^{(b)} - \langle \mathsf{ct}_j^{(a)}, \mathbf{s} \rangle = e_j + \Delta \cdot m_j$.

Remark 2 (Ring structure)**.** *The n vectors $\{\mathbf{a}_j\}$ are rows of the* negacyclic matrix *of $a(X)$:*

$$\mathbf{M}[j][k] = \begin{cases} a[j-k] & k \leq j \\ -a[n+j-k] & k > j \end{cases} \tag{2}$$

This $n \times n$ matrix is fully determined by the single polynomial $a(X)$. When $a(X)$ is derived from a shared seed, the server can reconstruct all $\mathbf{a}$-vectors without any communication. This structure is exploited by InspiRING: because the $\mathbf{a}$-vectors of all n LWE ciphertexts in one group share a single generating polynomial, the offline phase precomputes gadget decompositions of automorphisms of $a(X)$ from the seed alone.

Algorithm 3 LWE.Enc$_b(s(X), i^*, \ell, \mathtt{seed})$: Generate LWE b-values.

Input: Secret key $s(X)$; target index $i^* \in \{0, \ldots, \ell-1\}$; length ℓ (multiple of n); CRS seed
Output: $\mathsf{ct}_{\mathrm{lwe}}^{(b)} \in \mathbb{Z}_q^{\ell}$ ▷ vector of ℓ scalars
$n_r \leftarrow \ell/n$
for $r = 0, \ldots, n_r - 1$ **do**
  $a_r(X) \leftarrow \mathsf{H}(\mathtt{seed} \,\|\, \mathtt{"rlwe"} \,\|\, r)$ ▷ CRS
  $e_r(X) \leftarrow \chi_{\mathrm{err}}$
  $m_r(X) \leftarrow \begin{cases} \Delta \cdot X^{i^* \bmod n} & \text{if } r = \lfloor i^*/n \rfloor \\ 0 & \text{otherwise} \end{cases}$
  $b_r(X) \leftarrow a_r(X) \cdot s(X) + e_r(X) + m_r(X) \pmod q$
  Append coefficients of $b_r(X)$ to $\mathsf{ct}_{\mathrm{lwe}}^{(b)}$
end for
return $\mathsf{ct}_{\mathrm{lwe}}^{(b)}$

4.4 Approximate Gadget Decomposition

Algorithm 4 Decomp(f): Approximate centered gadget decomposition.

Input: Polynomial $f \in \mathcal{R}_q$
Output: Polynomials $\delta_1, \ldots, \delta_{d-1}$ with coefficients in $[-B/2, B/2)$
for each coefficient c of f **do** ▷ independent per coefficient
  $r \leftarrow c$
  for $i = 0, \ldots, d-1$ **do**
    $\delta_i \leftarrow r \bmod B$ ▷ remainder in $[0, B)$
    if $\delta_i \geq B/2$ **then** $\delta_i \leftarrow \delta_i - B$; $r \leftarrow r + B$
    end if ▷ center to $[-B/2, B/2)$
    $r \leftarrow (r - \delta_i)/B$ ▷ exact division; propagate carry
  end for
end for
return $(\delta_1, \ldots, \delta_{d-1})$ ▷ drop δ_0; residual $|\delta_0| \leq B/2$ is negligible noise

Decomposition: $f = \sum_{i=0}^{d-1} \delta_i \cdot B^i$ exactly (centered, with carries). Returned digits $\delta_1, \ldots, \delta_{d-1}$ satisfy $|\delta_i| \leq B/2$ ($4\times$ noise reduction vs. non-centered $|\delta_i| < B$). Dropping δ_0 saves one polynomial multiplication per use; the residual $|\delta_0| \leq B/2 \approx 2^{17}$ is negligible vs. $\Delta/2 \approx 2^{37}$.

4.5 Automorphism

The automorphism $\tau_k : \mathcal{R}_q \to \mathcal{R}_q$ substitutes $X \mapsto X^k$, i.e.:

$$\tau_k(f(X)) = f(X^k) \bmod (X^n + 1)$$

Concretely, τ_5 maps $s(X) \mapsto s(X^5)$, and τ_{2n-1} maps $s(X) \mapsto s(X^{-1})$.

Algorithm 5 $\tau_k(f(X))$: Apply automorphism (k odd, $\gcd(k, 2n) = 1$).

Input: $f(X) = \sum_j f_j X^j \in \mathcal{R}_q$
Output: $\tau_k(f)(X) = f(X^k) \bmod (X^n + 1)$
Allocate $h[0\,..\,n-1] \leftarrow 0$
for $j = 0, \ldots, n-1$ **do**
    $j' \leftarrow j \cdot k \bmod 2n$
    if $j' < n$ **then** $h[j'] \leftarrow h[j'] + f_j$
    else $h[j'-n] \leftarrow h[j'-n] - f_j$ ▷ $X^n \equiv -1$
    end if
end for
return $h(X)$

4.6 Seed Expansion

All CRS-derived random polynomials use a deterministic hash:

$$a_i(X) = \mathsf{H}(\mathtt{seed} \,\|\, i) \in \mathcal{R}_q \tag{3}$$

where $\mathsf{H} : \{0,1\}^* \to \mathcal{R}_q$ expands to n uniform values in $\mathbb{Z}_q$ by hashing with SHAKE-256 (or any XOF) in counter mode, reducing each output block modulo each RNS prime. The result is in **coefficient form**; the caller NTT-transforms as needed. The seed is a 64-byte ($\lambda = 512$ bit) value agreed between client and server.

4.7 RGSW Encryption and External Product

An RGSW ciphertext of $\mu(X) \in \mathcal{R}_q$ under key $s(X)$ consists of $2(d-1)$ RLWE-like ciphertexts ($(d-1 = 2)$ effective digits). Each row is $(\mathsf{ct}^{(a)},\ \mathsf{ct}^{(b)}) = (a(X),\ a(X) \cdot s(X) + e(X) + \text{message})$ with **no Δ scaling**:

$$\mathsf{RGSW}(\mu(X))^{(b)} = \begin{pmatrix} a_1^{(0)}(X)s(X) + e_1^{(0)}(X) - s(X)B^1\mu(X) & \cdots \\ a_1^{(1)}(X)s(X) + e_1^{(1)}(X) + B^1\mu(X) & \cdots \end{pmatrix} \tag{4}$$

The indices run from 1 to $d-1$ (digit 0 is dropped). The a-parts are CRS-derived; only the b-parts are transmitted.

Algorithm 6 RGSW.Enc$_b(s(X),\ \mu(X),\ \mathtt{seed})$: Generate RGSW b-parts.

Input: Secret key $s(X)$; message $\mu(X) \in \mathcal{R}_q$; CRS seed
Output: $\mathsf{ct}_{\text{rgsw}}^{(b)}$: $2(d-1)$ polynomials in $\mathcal{R}_q$ (NTT form)
for $j = 1, \ldots, d-1$ **do** ▷ $d-1$ effective digits
    $a_j^{(0)}(X) \leftarrow \mathsf{H}(\mathtt{seed} \,\|\, \mathtt{"rgsw0"} \,\|\, j)$; $e_j^{(0)}(X) \leftarrow \chi_{\text{err}}$ ▷ CRS
    $a_j^{(1)}(X) \leftarrow \mathsf{H}(\mathtt{seed} \,\|\, \mathtt{"rgsw1"} \,\|\, j)$; $e_j^{(1)}(X) \leftarrow \chi_{\text{err}}$
    $\mathsf{ct}_{\text{rgsw}}^{(b)}[0][j] \leftarrow a_j^{(0)}(X) \cdot s(X) + e_j^{(0)}(X) - s(X) \cdot B^j \cdot \mu(X) \pmod q$ ▷ top row; no Δ
    $\mathsf{ct}_{\text{rgsw}}^{(b)}[1][j] \leftarrow a_j^{(1)}(X) \cdot s(X) + e_j^{(1)}(X) + B^j \cdot \mu(X) \pmod q$ ▷ bottom row
end for
return $\mathsf{ct}_{\text{rgsw}}^{(b)}$

RGSW query monomial. For the second-dimension query targeting evaluation index j^*, the message is $\mu(X) = X^{j^* \cdot n/D}$, the evaluation point ω^{j^*} where $\omega(X) = X^{n/D}$ is the D-th root of unity in $\mathcal{R}_p$. The Horner evaluation at this point selects the j^*-th interpolation slice.

Algorithm 7 ExtProd($\mathsf{RGSW}(\mu)$, ct): RGSW $\times$ RLWE (approximate).

Input: $\mathsf{RGSW}(\mu)$ ($2(d-1)$ RLWE cts); $\mathsf{ct} = (a(X), b(X))$
Output: RLWE ciphertext encrypting $\mu(X) \cdot m(X)$
$(\delta_1^{(a)}, \ldots, \delta_{d-1}^{(a)}) \leftarrow \textsc{Decomp}(a(X))$ ▷ Algorithm 4
$(\delta_1^{(b)}, \ldots, \delta_{d-1}^{(b)}) \leftarrow \textsc{Decomp}(b(X))$ ▷ Algorithm 4
$\mathsf{resp} \leftarrow \sum_{j=1}^{d-1} \delta_j^{(a)} \cdot \mathsf{RGSW}[0][j] + \sum_{j=1}^{d-1} \delta_j^{(b)} \cdot \mathsf{RGSW}[1][j]$ ▷ $2(d-1)$ poly mults
return resp

5 InspiRING Packing

InspiRING packs n scalar b-values into one RLWE ciphertext. There are three entry points:

- $\textsc{KskGen}_b$ (client): generate b-parts of two key-switching keys.
- InspiRINGOffline (server, offline): precompute a-path aggregates and collapse.
- InspiRINGOnline (server, online): b-side collapse.

5.1 Key-Switching Key Generation

A key-switching key for index k enables the server to switch from $s(X^k)$ to $s(X)$. Each key has an a-part (from CRS) and a b-part (query-dependent, transmitted).

a-part (CRS-derived, server-side).

$$\mathsf{ksk}_k^{(a)}[j] = \mathsf{H}(\mathtt{seed} \,\|\, \mathtt{"ksk"} \,\|\, k \,\|\, j), \qquad j = 1, \ldots, d-1 \tag{5}$$

Algorithm 8 $\textsc{KskGen}_b(s(X), k, \mathtt{seed})$: Generate key-switching key b-part.

Input: Secret key $s(X)$; automorphism index k; CRS seed
Output: $\mathsf{ksk}_k^{(b)}$: $d-1$ polynomials in $\mathcal{R}_q$ (NTT form)
for $j = 1, \ldots, d-1$ **do** ▷ $d-1$ rows (approximate gadget)
  $a_j(X) \leftarrow \mathsf{H}(\mathtt{seed} \,\|\, \mathtt{"ksk"} \,\|\, k \,\|\, j)$ ▷ CRS
  $e_j(X) \leftarrow \chi_{\text{err}}$
  $\mathsf{ksk}_k^{(b)}[j](X) \leftarrow a_j(X) \cdot s(X) + e_j(X) + s(X^k) \cdot B^j \pmod q$ ▷ $s(X^k) = \tau_k(s(X))$
end for
return $\mathsf{ksk}_k^{(b)}$ ▷ $d-1$ polynomials

b-part (client-generated, transmitted). Two key-switching keys are needed: ksk_5 for rotation ($s(X) \to s(X^5)$), and ksk_{-1} for conjugation ($s(X) \to s(X^{-1})$). On the wire: $2(d-1)$ polynomials (b-parts only).

5.2 Offline Phase

Algorithm 9 InspiRINGOffline($\mathbf{a}_0, \ldots, \mathbf{a}_{n-1}$, seed)

Input: n LWE a-vectors $\mathbf{a}_j \in \mathbb{Z}_q^n$; CRS seed
Output: $\mathsf{precomp} = \{a(X),\ \mathbf{D}_+,\ \mathbf{D}_-,\ \mathbf{D}_{\text{final}}\}$

$(\mathsf{K}_+^{(a)},\ \mathsf{K}_-^{(a)},\ \mathsf{ksk}_{-1}^{(a)}) \leftarrow \text{ExpandKSK}_a(\mathtt{seed})$ ▷ Algorithm 10
$\hat{\mathbf{a}}_{\text{agg}}(X) \leftarrow \text{RingEmbed}(\mathbf{a}_0, \ldots, \mathbf{a}_{n-1})$ ▷ Algorithm 11
$(a(X),\ \mathbf{D}_+,\ \mathbf{D}_-,\ \mathbf{D}_{\text{final}}) \leftarrow \text{CollapseA}(\hat{\mathbf{a}}_{\text{agg}}(X),\ \mathsf{K}_+^{(a)},\ \mathsf{K}_-^{(a)},\ \mathsf{ksk}_{-1}^{(a)})$ ▷ Algorithm 12
return $\mathsf{precomp} = \{a(X),\ \mathbf{D}_+,\ \mathbf{D}_-,\ \mathbf{D}_{\text{final}}\}$

Remark 3 (Amortizing ExpandKSK$_a$)**.** *ExpandKSK$_a$ (Algorithm 10 below) depends only on the CRS seed, so its output is identical across every InspiRINGOffline call. In practice, expand once, store* $(\mathsf{K}_+^{(a)}, \mathsf{K}_-^{(a)}, \mathsf{ksk}_{-1}^{(a)})$*, and reuse it across multiple invocations to amortize the cost.*

Algorithm 10 ExpandKSK$_a$(seed): Derive and expand KSK a-parts from seed.

Input: CRS seed
Output: $\mathsf{K}_+^{(a)},\ \mathsf{K}_-^{(a)},\ \mathsf{ksk}_{-1}^{(a)}$

for $j = 1, \ldots, d-1$ **do** ▷ derive ksk_5 a-parts from seed via (5)
  $\mathsf{ksk}_5^{(a)}[j](X) \leftarrow \mathsf{H}(\mathtt{seed} \,\|\, \mathtt{"ksk"} \,\|\, 5 \,\|\, j)$
  $\mathsf{ksk}_{-1}^{(a)}[j](X) \leftarrow \mathsf{H}(\mathtt{seed} \,\|\, \mathtt{"ksk"} \,\|\, (2n-1) \,\|\, j)$
end for

for $k = 0, \ldots, n/2-2$ **do** ▷ both caches automorph the same ksk_5
  for $j = 1, \ldots, d-1$ **do**
    $\mathsf{K}_+^{(a)}[k][j](X) \leftarrow \tau_{5^k}(\mathsf{ksk}_5^{(a)}[j](X))$ ▷ switches $s(X^{5^{k+1}}) \to s(X^{5^k})$
    $\mathsf{K}_-^{(a)}[k][j](X) \leftarrow \tau_{2n-5^k}(\mathsf{ksk}_5^{(a)}[j](X))$ ▷ switches $s(X^{-5^{k+1}}) \to s(X^{-5^k})$
  end for
end for
return $(\mathsf{K}_+^{(a)},\ \mathsf{K}_-^{(a)},\ \mathsf{ksk}_{-1}^{(a)})$

Algorithm 11 RingEmbed($\mathbf{a}_0, \ldots, \mathbf{a}_{n-1}$): Embed and aggregate n LWE a-vectors.

Input: n LWE a-vectors $\mathbf{a}_j \in \mathbb{Z}_q^n$
Output: n aggregate polynomials $\hat{\mathbf{a}}_{\text{agg}}[0](X), \ldots, \hat{\mathbf{a}}_{\text{agg}}[n-1](X)$

▷ **Transform** (following Algorithm 1 of [1])
for $j = 0, \ldots, n-1$ **do**
  $\tilde{a}_j(X) \leftarrow n^{-1} \sum_{i=0}^{n-1} \mathbf{a}_j[i] \cdot X^{-i}$ ▷ $n \cdot n^{-1} = 1 \pmod q$; $X^{-i} = -X^{n-i}$
  for $k = 0, \ldots, n/2-1$ **do**
    $\hat{\mathbf{a}}_j[k](X) \leftarrow \tau_{5^k}(\tilde{a}_j(X))$ ▷ forward
    $\hat{\mathbf{a}}_j[k+n/2](X) \leftarrow \tau_{2n-5^k}(\tilde{a}_j(X))$ ▷ conjugate
  end for
end for

▷ **Aggregate** (shift by X^j and sum)
for $k = 0, \ldots, n-1$ **do**
  $\hat{\mathbf{a}}_{\text{agg}}[k](X) \leftarrow \sum_{j=0}^{n-1} \hat{\mathbf{a}}_j[k](X) \cdot X^j$
end for
return $\hat{\mathbf{a}}_{\text{agg}}(X)$ ▷ vector of n polynomials

Algorithm 12 CollapseA($\hat{\mathbf{a}}_{\text{agg}}(X)$, $\mathsf{K}_+^{(a)}$, $\mathsf{K}_-^{(a)}$, $\mathsf{ksk}_{-1}^{(a)}$): Full a-side Collapse.

Input: n aggregate polynomials $\hat{\mathbf{a}}_{\text{agg}}(X)$; expanded keys $\mathsf{K}_+^{(a)}$, $\mathsf{K}_-^{(a)}$ from Algorithm 10; raw $\mathsf{ksk}_{-1}^{(a)}$
Output: $a(X)$; gadget digit vectors $\mathbf{D}_+$, $\mathbf{D}_-$, $\mathbf{D}_{\text{final}}$

▷ **CollapseHalf** (+ direction): reduce $\hat{\mathbf{a}}_{\text{agg}}[0{:}n/2]$ under $s(X^{5^k}) \to s(X)$
for $k = n/2-1$ **down to** 1 **do**
  $\mathbf{D}_+[k] \leftarrow \text{Decomp}(\hat{\mathbf{a}}_{\text{agg}}[k](X))$
  for $j = 1, \ldots, d-1$ **do**
    $\hat{\mathbf{a}}_{\text{agg}}[k-1](X) \mathrel{-}= \mathbf{D}_+[k][j](X) \cdot \mathsf{K}_+[k-1][j](X)$
  end for
end for

▷ **CollapseHalf** (− direction): reduce $\hat{\mathbf{a}}_{\text{agg}}[n/2{:}n]$ under $s(X^{-5^k}) \to s(X^{-1})$
for $k = n/2-1$ **down to** 1 **do**
  $\mathbf{D}_-[k] \leftarrow \text{Decomp}(\hat{\mathbf{a}}_{\text{agg}}[n/2+k](X))$
  for $j = 1, \ldots, d-1$ **do**
    $\hat{\mathbf{a}}_{\text{agg}}[n/2+k-1](X) \mathrel{-}= \mathbf{D}_-[k][j](X) \cdot \mathsf{K}_-[k-1][j](X)$
  end for
end for

▷ **Final CollapseOne**: key-switch $\hat{\mathbf{a}}_{\text{agg}}[n/2]$ from $s(X^{-1})$ to $s(X)$
$\mathbf{D}_{\text{final}} \leftarrow \text{Decomp}(\hat{\mathbf{a}}_{\text{agg}}[n/2](X))$
$a(X) \leftarrow \hat{\mathbf{a}}_{\text{agg}}[0](X) - \sum_{j=1}^{d-1} \mathbf{D}_{\text{final}}[j](X) \cdot \mathsf{ksk}_{-1}^{(a)}[j](X)$
return $(a(X), \mathbf{D}_+, \mathbf{D}_-, \mathbf{D}_{\text{final}})$

Offline cost per group: n^2 automorphisms + n^2 polynomial multiply-accumulates (RingEmbed) + $2(n/2-1)+1$ gadget decompositions + $(2(n/2-1)+1)(d-1)$ multiply-accumulates (CollapseA). In the reference implementation, RingEmbed is fused into a single pass per share, giving $O(n^2)$

NTT multiply-accumulates total.

Implementing automorphisms. The Galois automorphism $\tau_t : f(X) \to f(X^t)$ in $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^n+1)$ is a coefficient-domain index permutation with sign flips. For $f(X) = \sum_i c_i X^i$, the result $g = \tau_t(f)$ has:

$$g[(i \cdot t) \bmod n] \mathrel{+}= \begin{cases} +c_i & \text{if } \lfloor i \cdot t/n \rfloor \text{ is even,} \\ -c_i & \text{if } \lfloor i \cdot t/n \rfloor \text{ is odd} \end{cases}$$

(the sign flip arises from the negacyclic reduction $X^n \equiv -1$). In our setting, the forward automorphisms use powers $5^0, 5^1, \ldots, 5^{n/2-1} \pmod{2n}$, and the conjugate automorphisms use $2n - 5^0, 2n - 5^1, \ldots, 2n - 5^{n/2-1}$. The conjugation automorphism τ_{2n-1} maps $X \to X^{-1}$.

5.3 Online Phase

The online phase uses $\mathbf{D}_+$, $\mathbf{D}_-$, $\mathbf{D}_{\text{final}}$ from the offline phase and the client's key-switching key b-parts. ExpandKSK$_b$ is called once; the expanded keys can be reused to ring-pack multiple groups of LWE ciphertexts.

Algorithm 13 ExpandKSK$_b(\mathsf{ksk}_5^{(b)})$: Expand KSK b-parts.

Input: b-part $\mathsf{ksk}_5^{(b)}$ ($d-1$ polynomials in $\mathcal{R}_q$)
Output: Expanded b-part caches $\mathsf{K}_+^{(b)}$, $\mathsf{K}_-^{(b)}$

for $k = 0, \ldots, n/2-2$ **do** ▷ both caches automorph the same $\mathsf{ksk}_5^{(b)}$
  for $j = 1, \ldots, d-1$ **do**
    $\mathsf{K}_+^{(b)}[k][j](X) \leftarrow \tau_{5^k}\big(\mathsf{ksk}_5^{(b)}[j](X)\big)$
    $\mathsf{K}_-^{(b)}[k][j](X) \leftarrow \tau_{2n-5^k}\big(\mathsf{ksk}_5^{(b)}[j](X)\big)$
  end for
end for
return $(\mathsf{K}_+^{(b)}, \mathsf{K}_-^{(b)})$

Algorithm 14 InspiRINGOnline($\mathsf{precomp}$, $\mathsf{K}_+^{(b)}$, $\mathsf{K}_-^{(b)}$, $\mathsf{ksk}_{-1}^{(b)}$, $b_0, \ldots, b_{n-1}$): Pack one group.

Input: $\mathsf{precomp}$ from Algorithm 9; $\mathsf{K}_+^{(b)}$, $\mathsf{K}_-^{(b)}$ from Algorithm 13; raw $\mathsf{ksk}_{-1}^{(b)}$; n scalars $b_j \in \mathbb{Z}_q$
Output: RLWE ciphertext $(a(X),\ b(X))$

Parse $\mathsf{precomp} = \{a(X),\ \mathbf{D}_+,\ \mathbf{D}_-,\ \mathbf{D}_{\text{final}}\}$
$b(X) \leftarrow \sum_{j=0}^{n-1} b_j \cdot X^j$ ▷ embed b-scalars into polynomial

▷ **CollapseHalf** (+ direction)
for $k = n/2 - 1$ **down to** 1 **do**
  for $j = 1, \ldots, d-1$ **do**
    $b(X)$ −= $\mathbf{D}_+[k][j](X) \cdot \mathsf{K}_+^{(b)}[k-1][j](X)$
  end for
end for

▷ **CollapseHalf** (− direction) — sequential: uses updated b
for $k = n/2 - 1$ **down to** 1 **do**
  for $j = 1, \ldots, d-1$ **do**
    $b(X)$ −= $\mathbf{D}_-[k][j](X) \cdot \mathsf{K}_-^{(b)}[k-1][j](X)$
  end for
end for

▷ **Final CollapseOne**
for $j = 1, \ldots, d-1$ **do**
  $b(X)$ −= $\mathbf{D}_{\text{final}}[j](X) \cdot \mathsf{ksk}_{-1}^{(b)}[j](X)$
end for
return $(a(X),\ b(X))$

Online cost per group: $2(n/2-1)(d-1) + (d-1)$ NTT-domain polynomial multiply-accumulates. **Key expansion:** $2(n/2-1)(d-1)$ automorphisms, computed once and reused across all groups.

6 Protocol

The protocol has three phases: **Setup**, **Preprocess** (server, offline), and **Online** (client + server).

6.1 Setup

Algorithm 15 SETUP(N, w)

Input: Number of database entries N; entry size w bytes
Output: Public parameters `pp`

`coeffs_per_entry` $\leftarrow$ coefficients used to encode one entry (encoding choice; see Section 9)
`db_rows` $\leftarrow$ `DEFAULT_DB_ROWS` ▷ fixed default 2^{15}; overridable per deployment (see Section 9)
`db_cols` $\leftarrow N \cdot$ `coeffs_per_entry` / `db_rows`
$D \leftarrow D_{\max}$ ▷ interpolation degree from noise budget (Section 8.3)
`seed` $\leftarrow$ sample public CRS seed
return `pp` $= (N, w, n, p, q, d, D,$ `coeffs_per_entry`, `db_rows`, `db_cols`, `seed`$)$

The public parameters `pp` are shared between client and server. All subsequent algorithms receive `pp` implicitly.

6.2 Preprocess

Algorithm 16 PREPROCESS(pp, $\mathtt{db}_{\text{raw}}$)

Input: Public parameters pp; raw database $\mathtt{db}_{\text{raw}}$ (N records of w bytes)
Output: Encoded database $\mathtt{db} \in \mathbb{Z}_p^{\mathtt{db\_rows} \times \mathtt{db\_cols}}$; InspiRING precomputation $\{\mathsf{precomp}_g\}_{g=0}^{n_p-1}$

▷ **Step 1: Encode the database**
$\mathtt{db} \leftarrow$ EncodeDatabase($\mathtt{db}_{\text{raw}}$) ▷ Algorithm 17

▷ **Step 2: Derive CRS polynomials for the a-side mat-vec**
$n_r \leftarrow \mathtt{db\_rows}/n$
for $r = 0, \ldots, n_r - 1$ **do**
  $a_r(X) \leftarrow \mathsf{H}(\mathtt{seed} \,\|\, \mathtt{"rlwe"} \,\|\, r)$ ▷ same CRS as LWE.Enc$_b$, Algorithm 3
  $\tilde{a}_r(X) \leftarrow \tau_{2n-1}(a_r(X))$ ▷ conjugation: $\tilde{a}_r[0] = a_r[0]$, $\tilde{a}_r[i] = -a_r[n-i]$ for $i \geq 1$
  Convert $\tilde{a}_r(X)$ to NTT form
end for

▷ **Step 3: a-side mat-vec** — produces n LWE a-vectors per group
for $g = 0, \ldots, n_p - 1$ **do**
  for $j = 0, \ldots, n - 1$ **do** ▷ n columns within group g
    $\mathsf{acc}(X) \leftarrow 0$ in NTT form
    for $r = 0, \ldots, n_r - 1$ **do**
      $f_{r,j}(X) \leftarrow \sum_{i=0}^{n-1} \mathtt{db}[rn + i][gn + j] \cdot X^i$ ▷ column $gn + j$, row block r
      Convert $f_{r,j}(X)$ to NTT form
      $\mathsf{acc} \mathrel{+}= f_{r,j} \circ \tilde{a}_r$ ▷ pointwise multiply in NTT, accumulate
    end for
    Convert $\mathsf{acc}(X)$ to coefficient form
    $\mathbf{a}_{g,j} \leftarrow (\mathsf{acc}[0], \ldots, \mathsf{acc}[n-1]) \in \mathbb{Z}_q^n$ ▷ LWE a-vector for column $gn + j$
  end for
end for

▷ **Step 4: Packing offline** — one InspiRINGOffline call per group
for $g = 0, \ldots, n_p - 1$ **do**
  $\mathsf{precomp}_g \leftarrow$ InspiRINGOffline($\mathbf{a}_{g,0}, \ldots, \mathbf{a}_{g,n-1}, \mathtt{seed}$) ▷ Algorithm 9
end for
return $(\mathtt{db}, \{\mathsf{precomp}_g\}_{g=0}^{n_p-1})$

Why the a-side reduces to a polynomial product. The naive a-side mat-vec would compute, for each output column j, an n-dimensional vector $\mathbf{a}_j[k] = \sum_i \mathtt{db}[i][j] \cdot \mathbf{a}_{\text{lwe}}[i][k]$, where $\mathbf{a}_{\text{lwe}}[i]$ is the implicit LWE a-vector at row i. Splitting $i = rn + i'$, the row block r uses the negacyclic matrix of $a_r(X)$ (Remark 2): $\mathbf{a}_{\text{lwe}}[rn+i'][k] = a_r[i'-k]$ for $k \leq i'$ and $-a_r[n+i'-k]$ for $k > i'$. A direct calculation shows that the k-th coefficient of the ring product $f_{r,j}(X) \cdot \tilde{a}_r(X)$ in $\mathcal{R}_q$ equals $\sum_{i'} f_{r,j}[i'] \cdot \mathbf{a}_{\text{lwe}}[rn+i'][k]$, so accumulating $\sum_r f_{r,j} \cdot \tilde{a}_r$ and reading off the n coefficients yields exactly $\mathbf{a}_{g,j}$. This replaces an $n_r \cdot n^2$ scalar mat-vec per output column with a single length-n NTT product per (r, j).

This precomputation depends on the database and the CRS seed (not on the client's query). The key-dependent Collapse (InspiRINGOnline, Algorithm 14) runs once the client sends $\mathsf{ksk}^{(b)}$ at query time.

Algorithm 17 EncodeDatabase($\texttt{db}_{\text{raw}}$): Pack raw bytes and apply polynomial interpolation encoding.

Input: Raw database $\texttt{db}_{\text{raw}}$ (N records of w bytes); parameters `db_rows`, `db_cols`, D from `pp`
Output: Encoded matrix $\texttt{db} \in \mathbb{Z}_p^{\texttt{db\_rows} \times \texttt{db\_cols}}$

▷ **Encode each entry as `coeffs_per_entry` coefficients in** $[0, p)$
$\texttt{db} \leftarrow \mathbf{0}$
for $e = 0, \ldots, N-1$ **do**
  $\texttt{row} \leftarrow e \bmod \texttt{db\_rows}$
  $\texttt{col\_start} \leftarrow \lfloor e / \texttt{db\_rows} \rfloor \cdot \texttt{coeffs\_per\_entry}$
  $(c_0, c_1, \ldots, c_{\texttt{coeffs\_per\_entry}-1}) \leftarrow \text{Encode}_p(\texttt{db}_{\text{raw}}[e])$ ▷ encoding scheme fixed by `pp`; see Section 9
  for $j = 0, \ldots, \texttt{coeffs\_per\_entry} - 1$ **do**
    $\texttt{db}[\texttt{row}][\texttt{col\_start} + j] \leftarrow c_j$
  end for
end for

▷ **Per-row inverse DFT across packing groups**
$n_p \leftarrow \texttt{db\_cols}/n$
$D_{\text{act}} \leftarrow \min(D,\ n_p)$ ▷ small-DB fallback: when $n_p < D$, encode at the smaller degree
if $D_{\text{act}} \leq 1$ **then**
  return db ▷ no interpolation needed
end if
$\texttt{num\_cts} \leftarrow n_p / D_{\text{act}}$
$D_{\text{inv}} \leftarrow D_{\text{act}}^{-1} \bmod p$
for $\texttt{row} = 0, \ldots, \texttt{db\_rows} - 1$ **do**
  for $\texttt{ct} = 0, \ldots, \texttt{num\_cts} - 1$ **do**
    ▷ Gather D_{act} ring elements: one length-n polynomial per packing group
    for $d = 0, \ldots, D_{\text{act}} - 1$ **do**
      for $k = 0, \ldots, n-1$ **do**
        $\texttt{pts}[d][k] \leftarrow \texttt{db}[\texttt{row}][\texttt{ct} \cdot D_{\text{act}} \cdot n + d \cdot n + k]$
      end for
    end for
    $\texttt{coeffs} \leftarrow \text{InverseDFT}(\texttt{pts},\ p)$ ▷ Algorithm 18; length-D_{act} DFT over $\mathcal{R}_p$
    for $d = 0, \ldots, D_{\text{act}} - 1$ **do**
      for $k = 0, \ldots, n-1$ **do**
        $\texttt{db}[\texttt{row}][\texttt{ct} \cdot D_{\text{act}} \cdot n + d \cdot n + k] \leftarrow D_{\text{inv}} \cdot \texttt{coeffs}[d][k] \bmod p$
      end for
    end for
  end for
end for
return db

Remark 4 (Choice of Encode_p)**.** *Two natural realizations of* Encode_p*:*

- Bit-packing. *Split the* $8w$*-bit input into chunks of* $\lfloor \log_2 p \rfloor$ *bits, each interpreted as a coefficient in* $[0, 2^{\lfloor \log_2 p \rfloor}) \subseteq [0, p)$*. Simple and efficient; uses* $\texttt{coeffs\_per\_entry} = \lceil 8w / \lfloor \log_2 p \rfloor \rceil$ *coefficients per entry. Wastes* $\log_2 p - \lfloor \log_2 p \rfloor$ *bits per coefficient.*
- Base-p digits. *Interpret the* $8w$*-bit input as an integer and write it in base* p*. Uses* $\texttt{coeffs\_per\_entry} = \lceil 8w / \log_2 p \rceil$ *coefficients per entry, recovering nearly all of* $\log_2 p$ *bits per coefficient. Requires big-integer arithmetic during encode/decode.*

The default (Section 9) is bit-packing; switch to base-p when the residual $\log_2 p - \lfloor \log_2 p \rfloor$ bits matter. $\textsc{Decode}_p$ (the inverse map) is applied during $\textsc{Extract}$ (Algorithm 22).

InverseDFT over $\mathcal{R}_p$. The polynomial interpolation encoding uses a Cooley–Tukey inverse DFT over $\mathcal{R}_p = \mathbb{Z}_p[X]/(X^n + 1)$. The "evaluation points" are powers of a primitive element $\omega(X) \in \mathcal{R}_p$ with $\omega^D = 1$ (i.e., ω is a D-th root of unity in the ring $\mathcal{R}_p$). Since D is a power of two and $D \mid n$ (both are powers of two), we can take $\omega(X) = X^{n/D}$: then $\omega^d = X^n \equiv -1 \pmod{X^n + 1}$. For even d: $\omega^{D/2} = X^{n/2}$, and $\omega^d = X^n = -1$, so ω is a $2d$-th root of unity, suitable for a length-D negacyclic DFT.

Algorithm 18 $\textsc{InverseDFT}(\texttt{pts}[0\,..\,D-1], p)$: Cooley–Tukey inverse DFT over $\mathcal{R}_p$.

Input: D ring elements $\texttt{pts}[j] \in \mathcal{R}_p$ (evaluation points); plaintext modulus p; d is a power of two
Output: D ring elements $\texttt{coeffs}[j] \in \mathcal{R}_p$ (polynomial coefficients)
$\omega(X) \leftarrow X^{n/d} \in \mathcal{R}_p$ ▷ D-th root of unity; $\omega^D \equiv -1$
$\texttt{buf}[j] \leftarrow \texttt{pts}[j]$ for $j = 0, \ldots, d-1$ ▷ bit-reverse permutation of indices

▷ Cooley–Tukey butterfly (in-place, inverse direction)
$m \leftarrow 1$ ▷ half-group size
while $m < D$ **do**
  $w \leftarrow \omega^{-d/(2m)} \pmod p$ ▷ twiddle factor: inverse root of unity
  $w_j \leftarrow 1$
  for $j = 0, \ldots, m-1$ **do**
    for $k = j, j + 2m, j + 4m, \ldots$ **do** ▷ stride $2m$
      $u \leftarrow \texttt{buf}[k]; \quad v \leftarrow \texttt{buf}[k+m] \cdot w_j \pmod p$ ▷ ring multiply in $\mathcal{R}_p$
      $\texttt{buf}[k] \leftarrow u + v \pmod p$
      $\texttt{buf}[k+m] \leftarrow u - v \pmod p$
    end for
    $w_j \leftarrow w_j \cdot w \pmod p$ ▷ ring multiply by twiddle
  end for
  $m \leftarrow 2m$
end while
return $\texttt{buf}$ ▷ the normalization by D^{-1} is applied by the caller

Remark 5. *The "multiply by twiddle" steps are ring multiplications in $\mathcal{R}_p$: multiplying a polynomial by $\omega^k(X) = X^{kn/d}$ is a negacyclic shift (rotate coefficients with sign flips), not a scalar multiply. For $D \le 256$ and $n = 2048$, each twiddle is a monomial $X^{kn/D}$ with $n/D \ge 8$, so the shift is coarse-grained and efficient.*

6.3 Query

Algorithm 19 QUERY(pp, idx)

Input: Public parameters pp; target index $\mathsf{idx} \in \{0, \ldots, N-1\}$
Output: Query state qst (kept by client for Algorithm 22); query qry (sent to server)

▷ **Index decomposition**
$i^* \leftarrow \mathsf{idx} \bmod \texttt{db\_rows}$ ▷ target row (first dimension)
$\mathsf{col} \leftarrow \lfloor \mathsf{idx} / \texttt{db\_rows} \rfloor \cdot \texttt{coeffs\_per\_entry}$ ▷ starting column of target entry
$j^* \leftarrow \lfloor \mathsf{col}/n \rfloor \bmod D$ ▷ evaluation point (Horner)
$k^* \leftarrow \lfloor \mathsf{col}/(n \cdot D) \rfloor$ ▷ response ciphertext index
$\ell^* \leftarrow \mathsf{col} \bmod n$ ▷ coefficient offset within ciphertext

▷ **KeyGen**
$s(X) \leftarrow \text{SKGEN}()$ ▷ Algorithm 1
$\mathsf{ksk}_5^{(b)} \leftarrow \text{KSKGEN}_b(s(X), 5, \texttt{pp.seed})$ ▷ Algorithm 8
$\mathsf{ksk}_{-1}^{(b)} \leftarrow \text{KSKGEN}_b(s(X), 2n-1, \texttt{pp.seed})$ ▷ Algorithm 8

▷ **Encrypt**
$\mathsf{ct}_{\text{lwe}}^{(b)} \leftarrow \text{LWE.ENC}_b(s(X), i^*, \texttt{pp.db\_rows}, \texttt{pp.seed})$ ▷ Algorithm 3
$\mathsf{ct}_{\text{rgsw}}^{(b)} \leftarrow \text{RGSW.ENC}_b(s(X),\ X^{j^* \cdot n/D},\ \texttt{pp.seed})$ ▷ Algorithm 6; evaluation point ω^{j^*}

$\mathsf{qst} \leftarrow (s(X),\ k^*,\ \ell^*)$
$\mathsf{qry} \leftarrow (\mathsf{ksk}_5^{(b)},\ \mathsf{ksk}_{-1}^{(b)},\ \mathsf{ct}_{\text{lwe}}^{(b)},\ \mathsf{ct}_{\text{rgsw}}^{(b)})$
return (qst, qry)

6.4 Answer

Algorithm 20 ANSWER(pp, qry, $\mathsf{precomp}$)

Input: Public parameters pp; query qry; $\mathsf{precomp}$ from Algorithm 16
Output: Response resp: c RLWE ciphertexts in $\mathcal{R}_q$
Unpack $\mathsf{qry} = (\mathsf{ksk}_5,\ \mathsf{ksk}_{-1},\ \mathsf{ct}_{\mathrm{lwe}}^{(b)},\ \mathsf{ct}_{\mathrm{rgsw}})$
▷ **Step 1: Matrix–vector product (b-side only; a-side was precomputed in Preprocess)**
for $j = 0, \ldots,$ `db_cols` $- 1$ **do** ▷ parallel over j
  $\tilde{\mathsf{ct}}_{\mathrm{lwe}}^{(b)}[j] \leftarrow \sum_i \mathtt{db}[i][j] \cdot \mathsf{ct}_{\mathrm{lwe}}^{(b)}[i] \pmod q$ ▷ `u16` $\times$ `u64`, per RNS limb
end for
▷ **Step 2: InspiRING packing** (combines precomputed a-path with online b-values)
$(\mathsf{K}_+^{(b)},\ \mathsf{K}_-^{(b)}) \leftarrow \text{EXPANDKSK}_b(\mathsf{ksk}_5^{(b)})$ ▷ Algorithm 13
$n_p \leftarrow$ `db_cols`$/n$
for $g = 0, \ldots, n_p - 1$ **do**
  $\mathsf{ct}_{\mathrm{rlwe}}[g] \leftarrow \text{INSPIRINGONLINE}(\mathsf{precomp}_g,\ \mathsf{K}_+^{(b)},\ \mathsf{K}_-^{(b)},\ \mathsf{ksk}_{-1}^{(b)},\ \tilde{\mathsf{ct}}_{\mathrm{lwe}}^{(b)}[gn\,..\,(g{+}1)n{-}1])$ ▷ Algorithm 14
end for
▷ **Step 3: Horner evaluation** (Algorithm 21)
$c \leftarrow n_p/D$
for $g = 0, \ldots, c - 1$ **do** ▷ parallel over g
  $\mathsf{resp}[g] \leftarrow \text{HORNEREVAL}(\mathsf{ct}_{\mathrm{rlwe}}[g \cdot D\,..\,(g{+}1)D{-}1],\ \mathsf{ct}_{\mathrm{rgsw}})$
end for
return $\mathsf{resp}[0\,..\,c{-}1]$

Algorithm 21 HORNEREVAL($\mathsf{ct}_{\mathrm{rlwe}}[0\,..\,D{-}1]$, $\mathsf{ct}_{\mathrm{rgsw}}$): Evaluate one interpolation polynomial via Horner's method.

Input: D packed RLWE ciphertexts $\mathsf{ct}_{\mathrm{rlwe}}[0\,..\,D{-}1]$; RGSW ciphertext $\mathsf{ct}_{\mathrm{rgsw}}$
Output: Single RLWE ciphertext

$\mathsf{acc} \leftarrow \mathsf{ct}_{\mathrm{rlwe}}[D-1]$ ▷ initialize with last ciphertext
for $i = D - 2$ **down to** 0 **do**
  $\mathsf{acc} \leftarrow \text{ExtProd}(\mathsf{ct}_{\mathrm{rgsw}},\ \mathsf{acc})$ ▷ Algorithm 7
  $\mathsf{acc} \leftarrow \mathsf{acc} + \mathsf{ct}_{\mathrm{rlwe}}[i] \pmod q$
end for
return acc

6.5 Extract

Algorithm 22 EXTRACT(pp, qst, resp)

Input: Public parameters pp; query state $\mathsf{qst} = (s(X), k^*, \ell^*)$; c RLWE ciphertexts
Output: Retrieved entry (w bytes)

$m(X) \leftarrow \text{RLWE.DEC}(s(X),\ \mathsf{resp}[k^*])$ ▷ Algorithm 2; k^* selects the polynomial group
$(c_0, \ldots, c_{\mathtt{coeffs\_per\_entry}-1}) \leftarrow (m[\ell^*], \ldots, m[\ell^* + \mathtt{coeffs\_per\_entry} - 1]) \in \mathbb{Z}_p^{\mathtt{coeffs\_per\_entry}}$
▷ ℓ^* is the coefficient offset
return $\text{DECODE}_p(c_0, \ldots, c_{\mathtt{coeffs\_per\_entry}-1})$ ▷ inverse of ENCODE_p from Algorithm 17; recovers w bytes

7 Serialization and Communication Cost

The protocol transmits a query (client → server) and a response (server → client). All values are bit-packed at the exact bit width (no byte-alignment padding). RLWE ciphertexts in the query use a *common reference string* (CRS/seed) for their $a(X)$ polynomials; only the $b(X)$ part is transmitted.

7.1 Query Serialization

Algorithm 23 SerializeQuery(qry)

Input: $\mathsf{qry} = (\mathsf{ksk}_5^{(b)}, \mathsf{ksk}_{-1}^{(b)}, \mathsf{ct}_{\text{lwe}}^{(b)}, \mathsf{ct}_{\text{rgsw}}^{(b)})$
Output: Bit-packed byte stream
▷ **Key-switching key b-parts**: $2(d-1)$ polynomials in NTT form
Bit-pack $\mathsf{ksk}_5^{(b)}$ and $\mathsf{ksk}_{-1}^{(b)}$: $2(d-1)$ polynomials, each $n \cdot \beta$ bits
▷ **LWE b-values**
Bit-pack $\mathsf{ct}_{\text{lwe}}^{(b)}$: `db_rows` scalars, each $\beta = 54$ bits
▷ **RGSW query b-parts**: $4(d-1)$ polynomials in NTT form
Bit-pack $\mathsf{ct}_{\text{rgsw}}^{(b)}$: $4(d-1)$ polynomials, each $n \cdot \beta$ bits

Algorithm 24 DeserializeQuery($\mathsf{pp}, \mathtt{bytes}$): Server-side.

Parse qry components from bit stream: $\mathsf{ksk}_5^{(b)}$, $\mathsf{ksk}_{-1}^{(b)}$, $\mathsf{ct}_{\text{lwe}}^{(b)}$, $\mathsf{ct}_{\text{rgsw}}^{(b)}$
Reconstruct a-parts from CRS via (5)
Assemble full keys: $\mathsf{ksk}_k = (\mathsf{ksk}_k^{(a)}, \mathsf{ksk}_k^{(b)})$; full RGSW similarly
return deserialized qry with full (a, b)-parts

7.2 Response Serialization (Modulus Switching)

Modulus switching compresses the response by reducing the ciphertext modulus. This is purely a serialization optimization: it trades noise budget for smaller wire size.

Algorithm 25 SerializeResponse(resp): Server-side, applied after Answer.

Input: c RLWE ciphertexts in $\mathcal{R}_q$
for $g = 0, \ldots, c-1$ **do**
  $(a(X), b(X)) \leftarrow \mathsf{resp}[g]$
  $b'[j] \leftarrow \lfloor b[j] \cdot q_0'/q \rceil$ for $j = 0, \ldots, n-1$ ▷ 20-bit coefficients
  $a'[j] \leftarrow \lfloor a[j] \cdot q_1'/q \rceil$ for $j = 0, \ldots, n-1$ ▷ 28-bit coefficients
  Serialize (a', b')
end for

Algorithm 26 DeserializeResponse(`bytes`): Client-side, applied before Extract.

for $g = 0, \ldots, c-1$ **do**
  Parse (a', b') from byte stream
  $b(X) \leftarrow \lfloor b'[j] \cdot q/q'_0 \rceil$ for each j ▷ rescale back to $\mathbb{Z}_q$
  $a(X) \leftarrow \lfloor a'[j] \cdot q/q'_1 \rceil$ for each j
  $\mathsf{resp}[g] \leftarrow (a(X), b(X))$
end for
return $\mathsf{resp}[0\,..\,c-1]$

7.3 Communication Cost Formulas

Let $\beta = \lceil \log_2 q \rceil$ and $(d-1)$ (approximate gadget digits). Let $\beta'_0 = \log_2 q'_0$ and $\beta'_1 = \log_2 q'_1$ (response coefficient widths).

Query cost.

$$|\mathsf{query}| = \underbrace{\texttt{db\_rows} \cdot \beta}_{b\text{-values}} + \underbrace{6(d-1) \cdot n \cdot \beta}_{\mathsf{ct}^{(b)}_{\mathrm{rgsw}}\ (4(d-1))\ +\ \mathsf{ksk}^{(b)}\ (2(d-1))} \quad \text{bits} \tag{6}$$

The second term is a fixed overhead independent of the database; only the b-values scale with the database.

Response cost.

$$|\mathsf{response}| = c \cdot n \cdot (\beta'_0 + \beta'_1) \quad \text{bits} \tag{7}$$

where $c = \texttt{db\_cols}/(n \cdot D)$ is the number of output ciphertexts.

Total.

$$|\mathsf{total}| = \texttt{db\_rows} \cdot \beta + 6(d-1) \cdot n \cdot \beta + \frac{\texttt{db\_cols}}{n \cdot D} \cdot n \cdot (\beta'_0 + \beta'_1) \tag{8}$$

Since $\texttt{db\_rows} \cdot \texttt{db\_cols} = N \cdot \texttt{coeffs\_per\_entry}$ (total coefficients in the encoded database), the first and third terms have an inverse relationship: increasing `db_rows` decreases `db_cols` (and thus c), and vice versa. The optimal `db_rows` (power of two) minimizes (8).

8 Error Analysis

8.1 Noise Sources

We track the noise $v(X) = b(X) - a(X) \cdot s(X) - \Delta \cdot m(X)$ throughout the protocol. Decryption succeeds when $\|v(X)\|_\infty < \Delta/2$.

Subgaussian convention. A zero-mean random variable Z is σ-subgaussian if $\mathbb{E}[\exp(tZ)] \leq \exp(\sigma^2 t^2/2)$ for all t. Tail bound: $\Pr[|Z| \geq u] \leq 2\exp(-u^2/(2\sigma^2))$. Key property (ring product): for fixed $c(X) \in \mathcal{R}_q$ and $e(X)$ with independent σ-subgaussian coefficients, each coefficient of $c(X) \cdot e(X)$ has subgaussian parameter $\leq \|c(X)\|_2 \cdot \sigma \leq \sqrt{n}\,\|c(X)\|_\infty \cdot \sigma$.

Remark 6 (Independence heuristic)**.** *The noise analysis assumes that noise contributions from different steps (encryption errors, key-switch errors, external-product errors) are* independent*. In practice, these terms share the same secret key s and may exhibit correlations through the ring structure. We follow the standard approach in the HE literature [5, 4]: treat the noise terms as independent subgaussians and sum their variances. This heuristic is well-validated empirically and is used by all major HE libraries for parameter selection.*

1. **Fresh encryption noise.** Each coefficient of $e(X)$ is σ-subgaussian ($\sigma = 3.2$).
2. **First-dimension accumulation.** For column j: $\mathsf{noise}_j = \sum_{i=0}^{\mathtt{db\_rows}-1} \mathtt{db}[i][j] \cdot e_i$. Each e_i is σ-subgaussian (independent); each $|\mathtt{db}[i][j]| \leq p$. By the subgaussian product rule, each term $\mathtt{db}[i][j] \cdot e_i$ is $(p \cdot \sigma)$-subgaussian. Summing $\mathtt{db\_rows}$ independent terms:

$$\sigma^2_{\text{first}} = \mathtt{db\_rows} \cdot p^2 \cdot \sigma^2 = d_0 \cdot n \cdot p^2 \cdot \sigma^2 \tag{9}$$

   where $d_0 = \mathtt{db\_rows}/n$. *Note:* the reference formula uses $d_0 \cdot p^2 \cdot \sigma^2$ (without the factor n) because the InspiRING packing distributes the accumulated noise across n coefficient slots. After packing, the per-coefficient variance is $d_0 \cdot p^2 \cdot \sigma^2$.
3. **Key switching (InspiRING Collapse).**
   Derivation. Each Collapse step performs $(d-1)$ polynomial multiplications of a gadget digit $\mathbf{T}[k][j]$ (coefficients bounded by $B/2$) with a ksk component (whose noise has σ-subgaussian coefficients).
   For the product $\mathbf{T}[k][j](X) \cdot e_j(X)$ in $\mathcal{R}_q$: the ring product property gives per-coefficient subgaussian parameter $\|\mathbf{T}[k][j](X)\|_2 \cdot \sigma$. For centered digits uniform on $[-B/2, B/2)$: $\|\mathbf{T}[k][j](X)\|_2^2 \leq nB^2/12$. Per-coefficient variance: $nB^2\sigma^2/12$ per digit.
   Over $(d-1)$ digits per step: $(d-1) \cdot nB^2\sigma^2/12$. Over $n/2$ steps:

$$\sigma^2_{\text{ks}} = \frac{(d-1) \cdot n^2 \cdot B^2 \cdot \sigma^2}{24} \tag{10}$$

   The forward and conjugate passes each contribute (10), but act on complementary halves of the coefficient space, so the per-coefficient variance is (10) (not doubled).
4. **RGSW external product (per Horner step).**
   Derivation. One external product decomposes both components $a(X)$ and $b(X)$ of the RLWE ciphertext into $(d-1)$ digits each ($2(d-1)$ total), multiplied by $2(d-1)$ RGSW rows. The per-external-product variance is:

$$\sigma^2_{\text{rgsw}} = \frac{(d-1) \cdot n \cdot B^2 \cdot \sigma^2}{6} \tag{11}$$

   (The $B^2/6$ factor is $2 \times nB^2/12$ summed over both ciphertext components, divided by n.)
   In practice, $\sigma_{\text{rgsw}} \ll \sigma_{\text{ks}}$ (key switching dominates).
5. **Modulus switching rounding** (Section 7.2). The round-trip $q \to q' \to q$ introduces rounding noise per coefficient:
   - Body (b): $\leq q/(2\,q'_0) \approx 2^{33}$.
   - Mask (a): $\leq q/(2\,q'_1) \approx 2^{25}$ per coefficient, amplified by $s(X)$ in decryption: total $\leq n \cdot q/(2\,q'_1) \approx 2^{36}$.

   Combined: $\leq 2^{33} + 2^{36} \approx 2^{36.2}$, consuming $\approx 56\%$ of $\Delta/2 \approx 2^{37}$. This is a *deterministic* (worst-case) bound, not subgaussian; it must be subtracted from the noise budget before allocating the remaining budget to the subgaussian computation noise.

8.2 Combined Noise Before Horner Evaluation

Theorem 1 (Pre-Horner noise variance)**.** *The noise variance per coefficient after the first dimension + InspiRING packing is the sum of* (9)*,* (10)*, and* (11)*:*

$$\hat{\sigma}^2_{\text{before}} = \underbrace{d_0 \cdot p^2 \cdot \sigma^2}_{\textit{first dim (9)}} + \underbrace{\frac{(d-1) \cdot n^2 \cdot B^2 \cdot \sigma^2}{24}}_{\textit{key switching (10)}} + \underbrace{\frac{(d-1) \cdot n \cdot B^2 \cdot \sigma^2}{6}}_{\textit{RGSW ext. prod. (11)}} \tag{12}$$

where $d_0 = \mathtt{db\_rows}/n$. The approximate gadget adds a bounded residual $\leq B/2$ per coefficient per step, which is negligible when $B/2 \ll \Delta/2$.

Numerical values (concrete instantiation, Section 9.3).

$$\sigma^2_{\text{first}} = 16 \cdot 65535^2 \cdot 3.2^2 \approx 2^{39.4} \qquad (\log_2\text{-std} \approx 19.7;\ d_0 = 16 \text{ for } \texttt{db\_rows} = 2^{15})$$
$$\sigma^2_{\text{ks}} = 2 \cdot 2048^2 \cdot (2^{18})^2 \cdot 3.2^2/24 \approx 2^{57.9} \qquad (\log_2\text{-std} \approx 28.9 \text{ — dominates})$$
$$\sigma^2_{\text{rgsw}} = 2 \cdot 2048 \cdot (2^{18})^2 \cdot 3.2^2/6 \approx 2^{48.8} \qquad (\log_2\text{-std} \approx 24.4)$$
$$\hat{\sigma}^2_{\text{before}} \approx 2^{57.9} \qquad \log_2 \hat{\sigma}_{\text{before}} \approx 28.9$$

8.3 Maximum Interpolation Degree

The noise budget must accommodate both the *deterministic* modulus-switching noise and the *subgaussian* computation noise. The modswitch noise is $v_{\text{ms}} \approx 2^{36.2}$ (Section 7.2). The subgaussian computation noise (failure probability $< 2^{-40}$) must satisfy $\hat{\sigma}_{\text{total}} \cdot \sqrt{80 \ln 2} + v_{\text{ms}} < \Delta/2$ (using the subgaussian tail bound $\Pr[|Z| \geq u] \leq 2\exp(-u^2/(2\sigma^2))$, which requires $u^2/(2\sigma^2) \geq 40 \ln 2$).

Effective subgaussian budget after deducting modswitch noise:

$$\hat{\sigma}_{\text{ub}} = \frac{\Delta/2 - v_{\text{ms}}}{\sqrt{80 \ln 2}} \quad \Longrightarrow \quad \log_2 \hat{\sigma}_{\text{ub}} \approx 32.9 \tag{13}$$

Each Horner step adds noise of the same order as $\hat{\sigma}_{\text{before}}$. After D steps, the total computation noise is $\approx \hat{\sigma}_{\text{before}} \cdot \sqrt{D}$.

$$\boxed{D_{\max} = \left\lfloor \left(\frac{\hat{\sigma}_{\text{ub}}}{\hat{\sigma}_{\text{before}}} \right)^2 \right\rfloor = \left\lfloor 2^{2(32.9-28.9)} \right\rfloor = \left\lfloor 2^{8.0} \right\rfloor = 264 \quad \text{(approximate centered gadget)}} \tag{14}$$

Table 3: $D_{\max}$ comparison (accounting for modulus-switching noise).

Strategy	$\log_2 \hat{\sigma}_{\text{before}}$	$D_{\max}$ (pre-ms)	$D_{\max}$ (with ms)	**Notes**
Non-centered, exact	30.2	230	44	Baseline
Non-centered, approx	29.9	345	66	Drop δ_0; saves $1/d$ mults
Centered, exact	29.2	921	176	$4\times$ noise reduction
Centered, approx	**28.9**	**1382**	**264**	**Default.** Best of both

We choose d as the largest power of two $\leq D_{\max}$ so that all dimensions remain powers of two. With $D_{\max} = 264$: $D = 256 = 2^8$. At the $\approx$16 GB/60 B target with the default $\texttt{db\_rows} = 2^{15}$, $n_{\text{packed}} = 128 < D$, so the encoder operates at $D_{\text{act}} = \min(D, n_{\text{packed}}) = 128$ (the small-DB fallback in Algorithm 17), giving $c = 1$ output ciphertext.

9 Parameter Selection

9.1 Parameter Choice Flow

Parameters are determined in three tiers:

Tier 1: Fixed by security and scheme design.

- $n = 2048$ (lattice dimension; 128-bit security with $\log_2 q \leq 54$)
- $p = 65535 = 3 \cdot 5 \cdot 17 \cdot 257$ (composite, but $p-1$ fits in 16 bits, so each encoded coefficient occupies 2 bytes; see Section 9.2)[1]

[1] The IDFT encoding (Algorithm 18) requires only $\gcd(D, p) = 1$ (for $D^{-1} \bmod p$) and p odd (so $\omega = X^{n/D}$ has order $2D$ in $\mathcal{R}_p$); both hold for $p = 65535$. No part of the scheme requires p prime. A 16-bit prime ($p = 65521$) would work equivalently but offers no practical gain.

- $\sigma = 3.2$ (discrete Gaussian standard deviation)
- Automorphism generator $= 5$
- Secret key: χ_{sk}: uniform ternary $\{-1, 0, +1\}$

Tier 2: Fixed by noise analysis. Given Tier 1, the following are determined by the error analysis (Section 8):

- $q = q_0 \cdot q_1$: maximize $\log_2 q$ subject to $\log_2 q \leq 54$ and both $q_i \equiv 1 \pmod{2n}$
- $d = 3$, $(d - 1 = 2)$ effective digits (approximate centered gadget)
- $B = \left\lceil 2^{\lceil \log_2 q/d \rceil} \right\rceil = 2^{18}$ (gadget base)
- $q'_0 = 2^{20}$, $q'_1 = 2^{28}$ (response compression; trades noise budget for wire size)
- $D_{\max} = 264$ (max interpolation degree after modulus-switch noise deduction)
- $D = 2^{\lfloor \log_2 D_{\max} \rfloor} = 256$ (largest power-of-two $\leq D_{\max}$)

Tier 3: Derived from database shape. Given N (number of entries) and w (entry bytes, ≥ 2):

1. $\textsc{Encode}_p$: bit-pack at $\lfloor \log_2 p \rfloor = 15$ bits per coefficient (Algorithm 17; switch to base-p digits if the residual bit matters).
2. `coeffs_per_entry` $= \lceil 8w / \lfloor \log_2 p \rfloor \rceil = \lceil 8w/15 \rceil$.
3. Total coefficients: $C = N \cdot$ `coeffs_per_entry`.
4. `db_rows` $=$ `DEFAULT_DB_ROWS` $= 2^{15}$ (fixed default; rationale below).
5. `db_cols` $= \lceil C/$`db_rows`$\rceil$ rounded up to a multiple of n.
6. $c = \max(1,$ `db_cols`$/(n \cdot D))$.

The implementation uses a **fixed default** `db_rows` $= 2^{15} = 32768$ (a tall geometry), *not* a communication-minimizing search over candidates. The reason: a smaller `db_rows` does minimize communication, but it inflates the precomputation ("hint") memory M_{precomp} (Equation (17)), which scales with `db_cols` $= C/$`db_rows` and is what actually bounds the deployable database size on a fixed-memory GPU. The tall default keeps M_{precomp} small at the cost of a larger query; raise or lower `db_rows` manually to trade query size against memory (Table 5).

9.2 Server Memory Cost

The server keeps two data structures resident in (GPU) memory: the encoded database, and the per-group InspiRINGOffline precomputation.

Encoded database. After the InverseDFT encoding (Algorithm 18), each coefficient of `db` takes a value in $[0, p-1] = [0, 65534]$, which fits in 16 bits. With the 15-bit bit-packing of Algorithm 17, each entry of w bytes occupies $\lceil 8w/15 \rceil$ coefficients, each stored as `u16`:

$$M_{\texttt{db}} \;=\; \texttt{db\_rows} \cdot \texttt{db\_cols} \cdot 2 \;=\; 2N \lceil 8w/15 \rceil \;\approx\; \tfrac{16}{15} Nw \quad \text{bytes.} \tag{15}$$

Compared to a raw byte-for-byte database (Nw), the overhead is $\approx 7\%$ from the 15-vs-16 bit slack (recoverable with the base-p packing of Algorithm 17's remark). This is a $\sim$1.87$\times$ memory reduction over the natural $p = 65537$ choice, which would have forced a 32-bit-per-coefficient layout.

Precomputation per group. Each group's $\mathsf{precomp}_g = \{a(X), \mathbf{D}_+, \mathbf{D}_-, \mathbf{D}_{\text{final}}\}$ holds

$$\begin{aligned} \#\text{polynomials per group} &= \underbrace{1}_{a(X)} + \underbrace{2 \cdot (n/2 - 1)(d-1)}_{\mathbf{D}_+ \text{ and } \mathbf{D}_-} + \underbrace{(d-1)}_{\mathbf{D}_{\text{final}}} \\ &= 1 + (d-1)(n-1). \end{aligned}$$

Each polynomial is stored in NTT form with 2 RNS limbs of n `u32` coefficients (both primes are $< 2^{28}$):

$$M_{\text{group}} \;=\; [1+(d-1)(n-1)] \cdot 2n \cdot 4 \;\approx\; 8(d-1)\,n^2 \quad \text{bytes.} \tag{16}$$

At $n = 2048$, $d = 3$: $M_{\text{group}} = 4095 \cdot 16{,}384 \approx 64$ MiB. The $a(X)$ and $\mathbf{D}_{\text{final}}$ terms together contribute only $3 \cdot 16$ KB, so the formula is dominated by the $(\mathbf{D}_+, \mathbf{D}_-)$ pair.

Total precomputation. With $n_{\text{packed}} = \texttt{db\_cols}/n$ groups,

$$M_{\text{precomp}} \;=\; n_{\text{packed}} \cdot M_{\text{group}} \;=\; \frac{\texttt{db\_cols}}{n} \cdot [1+(d-1)(n-1)] \cdot 2n \cdot 4 \;\approx\; 8(d-1)\,n \cdot \texttt{db\_cols} \quad \text{bytes.} \tag{17}$$

This scales *linearly with* `db_cols`: increasing `db_rows` (which decreases `db_cols`) saves precomputation memory at the cost of larger query.

Total server memory.

$$M_{\text{server}} \;=\; M_{\texttt{db}} + M_{\text{precomp}} \;\approx\; 2\,\texttt{db\_rows} \cdot \texttt{db\_cols} + 8(d-1)\,n \cdot \texttt{db\_cols}. \tag{18}$$

For typical parameters ($d-1 = 2$, $n = 2048$), the precomp term is $32{,}768 \cdot \texttt{db\_cols}$ bytes and the database term is $2 \cdot \texttt{db\_rows} \cdot \texttt{db\_cols}$ bytes; precomp dominates whenever $\texttt{db\_rows} < 4(d-1)\,n = 16{,}384$.

Trade-off vs. communication. Together with the communication formula (8):

- Small `db_rows`: smaller query, more groups, large M_{precomp}.
- Large `db_rows`: bigger query, fewer groups, small M_{precomp}.

The Tier 3 selection in Section 9.1 only minimizes communication; for memory-constrained deployments, `db_rows` should be raised until M_{precomp} fits available device memory, even at the cost of a larger query.

9.3 Concrete Instantiation ($\approx$16 GB / 60 B)

The headline target keeps the database storage close to 16 GiB and chooses the entry size w for clean alignment under 15-bit packing. With $w = 60$ bytes per entry, $\lceil 8w/15 \rceil = 32$ coefficients per entry exactly, so `db_cols` is a power of two and no zero-padding is wasted. The raw database is $N \cdot w = 2^{28} \cdot 60 = 15$ GiB; the encoded `db` is exactly 16 GiB (the 16/15 inflation from 15-bit packing). A more conventional $w = 64$ entry size is supported but inflates `db_cols` by $\approx 9.4\%$ (35/32 coefficients per entry, non-power-of-two), unless the tighter base-p packing from the remark in Algorithm 17 is used (recovers 32 coefficients per 64 B entry).

Note: "16 GB" is approximate. Raw input is 15 GiB and the encoded `db` is 16 GiB exactly because every coefficient lives in $\mathbb{Z}_p$ with $p = 65535$ rather than holding a full 16 bits. The fixed default $\texttt{db\_rows} = 2^{15}$ needs 24 GiB of GPU memory, fitting comfortably on the target RTX 5090 (32 GB). Table 5 shows the full trade-off: a smaller `db_rows` cuts communication but the precomputation grows past device memory (reaching 80 GiB at the communication-optimal $\texttt{db\_rows} = 2^{12}$), while a larger `db_rows` shrinks memory further at the cost of a bigger query.

10 Implementation Notes

10.1 RNS / CRT Arithmetic

With $q = q_0 \cdot q_1$ (two NTT-friendly primes, both $< 2^{28}$), all polynomial arithmetic splits into two independent limbs. The first-dimension dot product multiplies `u16` DB entries by `u64` query values per limb. Each limb product is $\leq 2^{28} \cdot 2^{16} = 2^{44}$; summing `db_rows` of them ($2^{15}$ at the default) gives $\leq 2^{59}$, which fits in `u64`.

Table 4: Concrete parameters for the ≈16 GB target.

Parameter	Value	Tier
n	2048	1 (security)
q	$133304321 \times 135135233$	2 (noise)
p	65535	1 (design)
σ	3.2	1 (security)
D	256	2 (noise)
q'_0, q'_1	$2^{20}, 2^{28}$	2 (noise/comm tradeoff)
N	$2^{28} = 268{,}435{,}456$	Input
w	60 bytes (raw = 15 GiB)	Input
`db_rows`	$2^{15} = 32768$	3 (fixed default)
`db_cols`	$2^{18} = 262{,}144$	3 (derived)
n_{packed}	128	3 (derived)
D_{act}	$\min(256, 128) = 128$	3 (derived)
c	1	3 (derived)
Query	378 KB	(6)
Response	12 KB	(7)
Comm. total	**390 KB**	(8)
$M_{\texttt{db}}$	16 GiB	(15)
M_{precomp}	8 GiB	(17)
Server memory	**24 GiB**	(18)

Table 5: Memory–communication trade-off for ≈16 GB / 60 B at varying `db_rows` (the 2^{15} row is the fixed default).

`db_rows`	`db_cols`	n_{packed}	c	Query	Total comm.	$M_{\texttt{db}}$	M_{server}
2^{12}	2^{21}	1024	4	188 KB	236 KB	16 GiB	80 GiB
2^{13}	2^{20}	512	2	216 KB	240 KB	16 GiB	48 GiB
2^{14}	2^{19}	256	1	270 KB	282 KB	16 GiB	32 GiB
2^{15}	**2^{18}**	**128**	**1**	**378 KB**	**390 KB**	**16 GiB**	**24 GiB**
2^{16}	2^{17}	64	1	594 KB	606 KB	16 GiB	20 GiB
2^{20}	2^{13}	4	1	7.4 MB	7.4 MB	16 GiB	≈ 16 GiB

10.2 GPU: CUDA Kernels

All polynomial arithmetic runs on the GPU via custom CUDA kernels. The NTT is the most critical kernel, as it dominates key switching and external product costs.

NTT kernel design. We implement a radix-2 Cooley–Tukey NTT with two phases:

1. **Global phase** (stages $\log_2 n$ down to $\log_2 S$): Each butterfly spans more elements than fit in shared memory. Threads read/write global memory with coalesced access patterns. One thread block handles one (polynomial, RNS limb) pair for each stage.
2. **Shared-memory phase** (stages $\log_2 S$ down to 1): Once the working set fits in shared memory ($S = 1024$ or 2048 elements), load a contiguous segment, perform all remaining butterfly stages in shared memory, write back. This eliminates global memory round-trips for the fine-grained stages.

Table 6: Other database configurations (same Tier 1/2 parameters; w chosen for clean 15-bit alignment).

DB	Entry	N	db_rows	n_{packed}	c	Total comm.	M_{db}	M_{server}
≈1 GB	240 B	2^{22}	2^{15}	8	1	390 KB	1 GiB	1.5 GiB
≈8 GB	60 B	2^{27}	2^{15}	64	1	390 KB	8 GiB	12 GiB
≈16 GB	60 B	2^{28}	2^{15}	128	1	390 KB	16 GiB	24 GiB

Modular arithmetic. Both primes fit in 28 bits. Coefficients are stored as native `u32`; products of two coefficients fit in `u64`; sums of $d-1=2$ such products fit in `u55`. We reduce a `u55` accumulator x modulo q_c using a precomputed Barrett constant $\mu_c = \lfloor 2^{55}/q_c \rfloor < 2^{29}$:

$$t \leftarrow ((x \gg 27) \cdot \mu_c) \gg 28, \qquad r \leftarrow x - t \cdot q_c, \qquad \text{up to two corrections } r \mathrel{-}= q_c.$$

This avoids the GPU's slow 64-bit integer division entirely and runs in a handful of `u32` multiplies and shifts. Add/Sub use `u32` with conditional subtraction.

Twiddle factors. At initialization, find a primitive $2n$-th root of unity $\psi_c \in \mathbb{Z}_{q_c}$ for each RNS prime (test successive small generators against $\psi_c^n \equiv -1 \pmod{q_c}$), then precompute the bit-reversed power table $\{\psi_c^{\text{br}(i)} \bmod q_c\}_{i=0}^{n-1}$ on the host and copy to the device once. One n-element `u32` array per RNS limb (16 KB total at $n = 2048$); resident in device global memory and read through L2/constant cache during NTTs.

Kernel fusion. Where possible, fuse consecutive operations into single kernel launches to avoid global memory round-trips:

- **NTT + element-wise multiply + INTT**: ring multiplication in one kernel.
- **Gadget decompose + NTT + multiply-accumulate**: key-switch step in one kernel (the bottleneck of InspiRING Collapse).
- **INTT + extract coefficients**: LWE extraction fused with INTT.

Table 7: CUDA kernel summary.

Kernel	Parallelism	Notes
NTT / INTT	1 block per (poly, limb)	2-phase: global + shared memory
Ring multiply	1 block per (poly, limb)	Fused NTT → elt-wise → INTT
First-dim dot prod	1 block per column	`u16` × `u32` per limb
Key-switch step	1 block per (step, limb)	Fused decomp + NTT + mul-acc
Gadget decompose	Grid over coefficients	Centered, carry propagation
External product	Per-ciphertext	Fuse $2(d-1)$ decomp + mul-acc
Modulus switch	Grid over coefficients	Round + rescale

11 Optimizations vs. Reference Implementation

The following optimizations are introduced in `inspire-gpu` compared to the reference Rust implementation built on `spiral-rs` [1, 2].

References

[1] R. Akhavan Mahdavi, S. Patel, J. Y. Seo, K. Yeo. InsPIRe: Communication-Efficient PIR with Silent Preprocessing. *IEEE S&P*, 2026. ePrint 2025/1352.

[2] S. Menon, D. J. Wu. Spiral: Fast, High-Rate Single-Server PIR via FHE Composition. *IEEE S&P*, 2022.

[3] S. Menon, D. J. Wu. YPIR: High-Throughput Single-Server PIR with Silent Preprocessing. *USENIX Security*, 2024.

[4] Microsoft SEAL (release 4.1). https://github.com/microsoft/SEAL, 2023.

[5] J. Fan, F. Vercauteren. Somewhat Practical Fully Homomorphic Encryption. ePrint 2012/144.

Table 8: Optimization summary.

Optimization	**Impact**	**Details**
Approx. centered gadget (default)	$D_{\max}$: 34 $\rightarrow$ 264; $1/d$ fewer poly mults	Centered digits ($[-B/2, B/2)$, 4$\times$ noise reduction) + drop least significant digit (residual $\leq B/2 \approx 2^{17}$ negligible). Reduces RGSW to $4(d-1) = 8$ cts, key-switching keys to $2(d-1) = 4$ cts (Algorithm 4).
Optimized prime selection	$\log_2 q = 54.00$ (vs. 52.97)	Primes 133304321 $\times$ 135135233 maximize q under the $\log_2 q \leq 54$ constraint. Gives +1 bit noise budget vs. reference primes 67043329 $\times$ 132120577.
$\sigma = 3.2$ (vs. 6.4)	2$\times$ noise reduction	Standard HE noise width. Reference used $\sigma = 6.4$; reducing to 3.2 halves all Gaussian noise terms while maintaining security (validated by lattice estimator for $n = 2048$).
CUDA kernels (32-bit Barrett)	GPU: native `u32` arithmetic	Both primes fit in 28 bits. All modular reductions use a precomputed Barrett constant ($\mu_c = \lfloor 2^{55}/q_c \rfloor$), avoiding 64-bit division. Fused NTT + key-switch kernels avoid global memory round-trips.
Coalesced row-major mat-vec	GPU: $\sim$ 480 GB/s on A40	Database transposed to row-major at preprocess; mat-vec kernel with one thread per output column achieves coalesced reads, within 1–6% of pure memory-read bandwidth (Section 10.2). Auto-dispatch falls back to a parallel-reduction variant when the column count is too small to saturate the GPU.
Lazy InspiRINGOnline	Eliminates ExpandKSK$_b$	The conjugate and forward halves' expanded keys $\mathsf{K}_+, \mathsf{K}_-$ are derived from the same $\mathsf{ksk}_5^{(b)}$ by NTT-domain index permutation. We permute on the fly inside the collapse kernel rather than materialize the expanded keys, saving $\approx$ 128 MB of device memory per query and $\approx$ 60–80 ms of host-side expansion.
Multi-group mega-kernel	n_p Collapses in one launch	All $n_p \cdot 4$ collapse half-kernels (forward/conjugate $\times$ two RNS limbs) launched as a single CUDA grid with `blockIdx.y` ranging over groups, eliminating per-group launch overhead.